

TECHNICAL ARCHITECTURE REPORT · SYSTEMS AND MACHINE LEARNING

Decoupled Multi-Model Pipeline for Grounded Meeting Summarization

Verbatim Engineering Systems Group · Technical Architecture Specification
Component Specifications, Inference Roles, and Inter-Stage Output Contracts

ABSTRACT: This report specifies the three-model machine learning architecture in Verbatim. We identify the speech-to-text model and the two distinct language models. We detail each model's operational scope, inference roles, and serialized data contracts between stages. Deterministic invariance guardrails and lexical support checks govern model outputs. This design prevents hallucinated decisions, phantom commitments, and ungrounded task delegations.

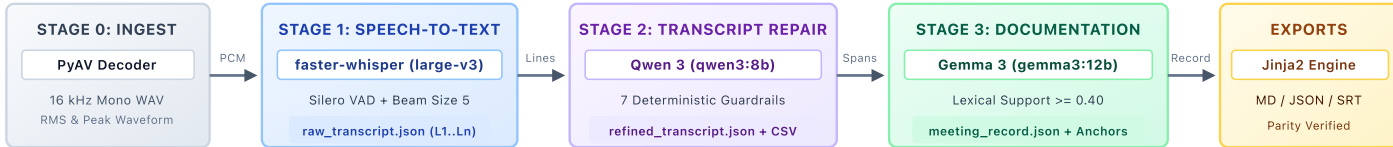


Figure 1: End-to-end dataflow pipeline with model assignments, safety guardrails, and intermediate serialized contracts.

1. MODEL IDENTIFICATION AND ROLES

Speech-to-Text: faster-whisper

STT ENGINE

Models: large-v3 (CUDA fp16) or distil-large-v3 (CPU int8).
Role: Decodes acoustic streams into sequential, timestamped segments. Silero VAD isolates active speech chunks. The model produces sub-second word boundaries and lexical confidence levels without cloud dependencies.

Language Model 1: Qwen 3 (Refiner)

LLM #1 · QWEN3:8B

Profiles: qwen3:8b (standard), qwen3:4b (lite), qwen3:14b (quality).
Role: Repairs acoustic mishearings, acronyms, and specialized engineering jargon. Runs in Ollama at temperature 0 with disabled thinking. It analyzes rolling 40-segment windows flanked by 4-segment read-only context. It proposes precise replacement spans rather than rewriting whole lines.

Language Model 2: Gemma 3 (Documenter)

LLM #2 · GEMMA3:12B

Profiles: gemma3:12b (standard, quality), gemma3:4b (lite).
Role: Synthesizes executive documentation across four decoupled stages: summary, chronological minutes, settled decisions, and action items. Operates in Ollama with context windows up to 32k tokens. It enforces evidence anchors back to transcript line identifiers.

2. INTER-STAGE OUTPUT PROGRESSION

Stage 1 → Stage 2: Raw Segment Emission

CONTRACT: RAW_TRANSCRIPT.JSON

The STT stage emits timestamped lines with monotonic IDs (L1, L2, ...). Each line carries start and end timestamps alongside word tokens. The backend saves this structure to disk and publishes line events via Server-Sent Events to the client.

Stage 2 → Stage 3: Guardrail-Gated Spans

CONTRACT: REFINED_TRANSCRIPT.JSON

Qwen returns discrete edit proposals in JSON: [segment_id, original, corrected]. Seven Python guardrails validate every span. Edits touching numbers, negations, or commitment verbs are blocked. Safe spans are applied deterministically to produce clean transcript text and corrections.csv.

Stage 3 → Export: Grounding and Parity

CONTRACT: MEETING_RECORD.JSON

Gemma extracts candidate items citing line ranges. A deterministic grounding filter enforces a 0.40 lexical support threshold against cited lines. Pronouns like "we" demote to Unspecified owners. Non-consensus points route to "Discussed, not settled". Outputs render to Markdown and JSON with automated parity validation.

PIPELINE STAGE	MODEL ENTITY	PROFILE DEFAULT	PRIMARY INPUT	GENERATED OUTPUT	VALIDATION BOUNDARY
Speech Recognition	faster-whisper	large-v3 (CUDA)	16 kHz WAV audio	raw_transcript.json	Silero VAD, hallucination repetition filter
Transcript Refiner	Qwen 3	qwen3:8b (8k ctx)	Windowed raw lines (L1..Ln)	Candidate replacement spans	7 invariance guardrails (numbers, negations)
Document Secretary	Gemma 3	gemma3:12b (16k ctx)	Refined lines (L1..Ln)	Structured record sections	Lexical support ≥ 0.40, pronoun demotion
Delivery & Export	Deterministic Code	Jinja2 Templates	Meeting record dict	Markdown, JSON, SRT, Zip	Bi-directional decision and task parity check